

GUQINAGENT: Tool-Interactive Performance-Action Planning for Jianpu-to-Jianzipu Generation

Anonymous Authors

Abstract

Jianzipu, the action-oriented notation of the Chinese guqin, jointly encodes strings, left-hand positions, right-hand strokes, ornaments, and stateful performance conventions. Generating it from a melodic score is therefore not mere text transduction: an output must agree with pitch, tuning, inherited state, and basic performability. We introduce GUQINAGENT, a tool-interactive formulation of constrained jianzipu generation. Our contributions are: (i) a full-score benchmark and executable runtime for iteratively editing a jianpu score into jianzipu; (ii) a warning-guided inference mechanism that feeds deterministic pitch and conservative playability diagnostics back to the agent without forcing a rewrite; (iii) a two-stage agent that separates pitch-grounded fingering from idiomatic guqin refinement; and (iv) targeted inverse trajectory construction that converts annotated scores into auditable, redacted tool-use supervision. Phrase partitioning is part of the generation algorithm, not an assumption on the input. The runtime exposes pitch candidates, stateful editing, compact context retrieval, and advisory warnings, and supports structured event-level evaluation rather than string overlap. We will release the model, redacted training trajectories, test benchmark, and executable runtime. This draft presents the task and system; final numerical results will be added after benchmark evaluation.

1 Introduction

Jianzipu is an open, compositional notation system that encodes how a guqin performance is produced, rather than merely which pitches sound. A notation unit can combine a string, hui position, left-hand finger, right-hand stroke, and ornament in one compact action description. Its interpretation is also stateful: harmonic mode remains active until terminated, walking gestures inherit positions, and compound gestures may cover multiple events. Similar text can thus imply different pitches, while different surface forms can be musically equivalent [2, 1].

Recent work has made important progress on the preceding digitization problem: recognizing jianzi from manuscript images and recovering their internal compositional structure, including zero-shot component-sequence recognition and full-page mixed-script extraction [2, 1]. Our problem begins after such recognition. Given a machine-readable jianpu melody and score metadata, the system must select an executable realization in jianzipu. This is not transcription: the input does not prescribe a unique string, hui, finger, or ornament. It is a constrained performance-action planning problem, where pitch agreement is necessary but not sufficient.

Existing text generation alone does not expose whether a candidate realizes the intended pitch under the current tuning, inherits harmonic or left-hand state correctly, or remains compatible with notation coverage rules. Recent language-model research has explored interleaving reasoning with external actions, API use, and iterative feedback, but those methods do not model guqin-specific tuning, notation state, or performance constraints [5, 4, 3]. We formulate generation as interactive full-score plan editing. The model observes a complete jianpu score, partitions it into context-aware work units, queries deterministic tools when useful, and incrementally writes one event-indexed final

notation plan. Pitch and state inconsistencies are exposed as actionable feedback, while idiomatic choices remain model decisions. In summary, our contributions are fourfold:

1. We introduce *constrained jianzipu generation*, a benchmark and executable runtime for reproducible score-to-jianzipu evaluation.
2. We introduce warning-guided inference: deterministic pitch and conservative notation-constraint diagnostics become visible, actionable feedback rather than hidden post-hoc validation.
3. We design GUQINAGENT, a two-stage Fingering→Guqinizer agent that distinguishes basic sound production from stylistic refinement.
4. We develop targeted inverse trajectory construction, which replays annotations through the runtime and exports public, redacted supervision.

2 Related Work

Jianzipu digitization. Recent systems address the visual recognition problem. JZ-Glyph predicts canonical component sequences for individual glyphs under zero-shot splits, while JZ-Tab performs end-to-end structured extraction from full-page mixed-script manuscripts [2, 1]. These systems recover notation from images. We take machine-readable jianpu as input and generate a playable jianzipu realization for a complete score, addressing the downstream performance-planning problem.

Tool-using and iterative language models. ReAct interleaves model reasoning and actions that query an external environment; Toolformer studies learning when and how to invoke APIs; Self-Refine iteratively improves an output using feedback [5, 4, 3]. GUQINAGENT shares the interactive and iterative setting, while grounding feedback in deterministic, domain-specific operations over pitch, tuning, event alignment, and inherited notation state. Its phrase partitioning and two-stage Fingering-to-Guqinizer workflow are designed for whole-score tablature generation.

3 Methodology

3.1 Task Formulation and Benchmark

Each input is a complete score $x = (m, e_{1:N})$: metadata m includes mode and tuning, while $e_{1:N}$ are all jianpu events in the score. Every event has a stable *event index*, source notation, duration, structural type, and target pitch when defined. The output is one full-score plan $y = (y_1, \dots, y_N)$ where y_i is jianzipu text or an explicit empty display value. Empty display values are not deletion commands; they can mean that a compound gesture or continuation covers a later event. ABC notation may be derived internally for pitch processing, but it is not a separate task input.

The task is stateful. Let $S_i = \mathcal{T}(S_{i-1}, y_i)$ be the deterministic notation-state transition. A plan must satisfy runtime invariants and is evaluated against an expert plan at structured-event level. We score pitch only where it is reliably resolvable, never treating unresolved notation as either correct or incorrect.

3.2 Whole-Score Phrase Planning

Directly editing an entire score in one context is impractical and obscures local musical decisions. GUQINAGENT therefore derives an ordered partition $\mathcal{P}(x) = (p_1, \dots, p_K)$ from the full score, using

Component	Contents
Input	Complete-score metadata, tuning, jianpu events, and stable event indices.
Output	One full-score, event-indexed jianzipu plan with explicit empty display values.
Runtime	Pitch candidates, plan editing, compact context lookup, validation, and audit.
Reference	Expert annotation, private to the offline teacher and evaluation pipeline.

Table 1: Benchmark representation. The student policy never receives the reference plan.

musical boundaries and bounded event ranges. For each p_k , the agent receives the current editable slice together with the state inherited from completed earlier slices. It may retrieve selected earlier slices when needed, then commits edits before the runtime advances the full-score state. Thus phrases are algorithmic work units that make whole-score generation tractable; they are neither independent benchmark examples nor externally supplied inputs.

The benchmark uses score-level splits. A high-confidence test subset retains scores with reference pitch-match rate strictly above 70% under the current 50-cent auditor, calculated over parseable reference events only. The current test inventory has 44 scores; 39 meet this condition. The other scores remain available for diagnostic analysis rather than being silently treated as clean gold pitch labels.

3.3 Executable Runtime

The same runtime is used for teacher generation and inference. This avoids a train–test mismatch where a model learns parser artifacts unavailable at deployment.

Formally, generation is a stateful, tool-interactive process. At interaction step t , the agent conditions on the score, the current work unit, the editable plan, the inherited notation state, and previous runtime observations:

$$h_t = (x, p_k, y^{(t)}, S^{(t)}, o_{<t}), \quad a_t \sim \pi_\theta(\cdot \mid h_t), \quad (1)$$

where $y^{(t)}$ and $S^{(t)}$ denote the full editable plan and its replayed notation state after step t , so that event-level notation and state remain y_i and S_i as defined in Section 3.1. The runtime executes each action deterministically and returns the updated plan, notation state, and an observable feedback signal:

$$(y^{(t+1)}, S^{(t+1)}, o_t) = \mathcal{R}(x, y^{(t)}, S^{(t)}, a_t), \quad (2)$$

where o_t may contain pitch candidates, plan previews, notation-state information, or conservative warnings. The policy observes the world only through this channel, so all deliberation remains grounded in executable feedback rather than in private annotation.

Pitch candidates. `get_pitch_candidates` accepts one or more event indices, groups equal target MIDI values, and returns candidate string–hui realizations. It uses normalized score tuning rather than assuming standard tuning, and represents open strings, stopped tones, harmonics, inherited positions, and supported compound gestures. Candidates carry exact, approximate, or unavailable confidence labels.

Stateful editing. The only editing action is `edit_plan(jianzi_rows)`, an event-indexed batch update. The runtime replays notation state and returns a compact preview. High-confidence pitch contradictions are warnings, not automatic rewrites: some advanced gestures cannot be resolved

from text alone. Replay state includes tuning, harmonic region entry and termination, inherited left-hand position, walking gestures, and compound-action coverage.

Warning-guided inference. Each `edit_plan` preview exposes warnings directly beside the affected event. The warning layer includes reliably parsed pitch mismatches, potentially stale harmonic-region state, implausible walk spans or directions, an open-string resonance immediately stopped on the same string, non-adjacent paired *bo/po* strokes, excessive same-finger reaches, and mechanically cramped low-hui stopped notes. These checks are deliberately conservative: ambiguous or compound notation is skipped rather than guessed. A warning revealed by an edit is carried into a later agent turn, where the agent can query pitch candidates and either submit a genuinely changed repair or retain the notation with a reason. Thus warnings guide deliberation without turning a partial parser into an automatic musical authority; repeated identical edits are stopped to avoid correction loops.

Context retrieval. `list_context` returns a compact section-grouped phrase index; targeted retrieval supplies only the relevant prior phrase. This prevents a full-score directory from consuming the model’s context window.

3.4 Two-Stage Guqin Agent

Fingering stage. For each derived work unit, Fingering edits the corresponding blank portion of the full-score plan into a pitch-grounded basic realization: string, hui, left-hand contact, and elementary right-hand stroke. It may batch-query pitch candidates and fills relevant events without gratuitous final-stage ornamentation.

Guqinizer stage. Guqinizer receives the accepted Fingering plan, not a blank string. It may preserve correct material or revise it with ornaments, walking gestures, harmonic notation, compound actions, and display compression. It has access to the same candidate tool and current-plan audit, so every revision remains inspectable.

This decomposition is an inductive bias, not a claim that basic and idiomatic notation have a unique boundary. We compare it with a single-stage policy that edits a blank plan directly to final notation under the same public tools.

3.5 Expert Artifact-Guided Inverse Trajectory Construction

Many specialized domains provide high-quality expert artifacts, such as scores, designs, protocols, or structured records. However, these artifacts typically describe the desired outcome rather than the interactive procedures required to obtain it. This creates a fundamental challenge for domain-specific agent training: how to transform expert artifacts into executable trajectories without using the artifacts as direct answer supervision.

We formulate trajectory construction as expert-guided inverse execution. Let z denote private expert-artifact guidance and $\mathcal{K}$ the retrieved private domain knowledge; both reach the teacher through a channel inaccessible to the student. The teacher produces an executable trajectory through the same public runtime $\mathcal{R}$ available at deployment:

$$\tau^T \sim \pi_T(\cdot \mid x, z, \mathcal{K}, \mathcal{R}), \quad (3)$$

where π_T must discover a valid sequence of tool interactions and plan edits leading to an executable solution, using the expert artifact as semantic guidance for exploring valid execution paths rather than

mechanically copying it. The student-visible trajectory is then obtained by removing private-source information:

$$\tilde{\tau} = g_{\text{redact}}(\tau^T), \quad (4)$$

such that every conditioning signal retained in $\tilde{\tau}$ —score context, public actions, tool outputs, and public reasoning—is available to the student policy at deployment time.

In our guqin setting, the private guidance contains phrase-level reference Jianzipu annotations. The teacher operates on the student-visible score context, invokes public tools, and updates an editable performance plan through executable interactions. All tool calls, tool responses, and validated intermediate or final plans are recorded from the runtime rather than generated as free-form text. Stage 1 focuses on constructing a playable basic fingering plan with correct pitch realization and notation, while Stage 2 performs idiomatic refinement by incorporating transitions, ornaments, and domain-specific performance conventions.

Domain semantic augmentation. Expert artifacts often contain compact symbols whose meanings cannot be fully recovered from surface forms alone. To bridge this gap, we maintain a curated domain knowledge base describing complex guqin techniques and their execution semantics. During trajectory generation, technique-relevant descriptions are retrieved according to symbols appearing in the current artifact and provided as private semantic guidance to the teacher. This knowledge explains the meaning and constraints of domain symbols rather than directly specifying target actions, allowing the teacher to reason about executable alternatives while avoiding direct answer imitation.

Trajectory validation and leakage control. Before trajectory construction, unreliable artifact segments are removed based on parsing quality and consistency checks. After generation, the resulting trajectories are validated through the execution runtime. Private teacher reasoning is transformed into public reasoning grounded only in observable score context, previous public messages, and tool outputs. The exported SFT data exclude private-reference fields and preserve only interactions accessible to the student agent during inference.

The resulting procedure can be summarized as follows:

1. Initialize a stage-specific editable plan from the current domain state.
2. Provide the teacher with observable context and private expert-artifact guidance.
3. Execute bounded public tool interactions and iteratively update the executable plan.
4. Validate the final trajectory, remove private-source information, and export the student-visible training example.

Given the resulting public trajectory dataset $\mathcal{D}$, we train the student with standard supervised fine-tuning:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{\tilde{\tau} \in \mathcal{D}} \sum_{u \in \mathcal{Y}(\tilde{\tau})} \log p_{\theta}(u \mid h_u^{\text{pub}}), \quad (5)$$

where $\mathcal{Y}(\tilde{\tau})$ denotes the supervised assistant tokens and h_u^{pub} contains only student-visible context. The teacher conditions on z and $\mathcal{K}$ when constructing τ^T , but the training context h_u^{pub} never contains them; this is the formal statement of reference-leakage control.

Method	Pitch Acc. $\uparrow$	MAE $\downarrow$	String $\uparrow$	LH F1 $\uparrow$	RH F1 $\uparrow$	Rules $\downarrow$
Direct	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]
Single-stage agent	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]
Two-stage GUQINAGENT	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]

Table 2: Results template for the high-confidence test subset. Final numbers will include denominators and unresolved-event counts.

Inference feedback	Pitch Acc. $\uparrow$	Rule Viol. $\downarrow$	Warning repair $\uparrow$	Repeated-edit stop $\downarrow$	Mean turns $\downarrow$	Expert playability $\uparrow$
Without warning feedback	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]
With warning feedback (GUQINAGENT)	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]	[TODO: -]

Table 3: Warning-guided inference ablation. Both rows use the same model, tools, decoding configuration, and maximum turn budget; only runtime warning feedback differs. “Warning repair” is the fraction of initially warned events that are later changed and no longer warned. “Repeated-edit stop” counts trajectories terminated after an identical edit submission. Expert playability is reported only if a blinded musician study is conducted.

4 Experimental Setup

4.1 Evaluation Metrics

We parse predictions and references into unified events containing source and event IDs, target pitch, string, left-hand position and finger, right-hand technique, tone type, ornaments, and necessary state. We do not use BLEU, ROUGE, or direct final-string equality as primary metrics. Let $\hat{q}_i$ and q_i^{ref} denote the pitches realized by the prediction and the reference at event i , let $d_c(\cdot, \cdot)$ measure their distance in cents, and let $\mathcal{I}_{\text{pitch}} = \{i : \hat{q}_i \text{ and } q_i^{\text{ref}} \text{ are both resolvable}\}$ be the jointly resolvable set. Our primary pitch metrics are

$$\text{PitchAcc}_{50} = \frac{1}{|\mathcal{I}_{\text{pitch}}|} \sum_{i \in \mathcal{I}_{\text{pitch}}} \mathbf{1}[d_c(\hat{q}_i, q_i^{\text{ref}}) \leq 50], \quad \text{PitchMAE} = \frac{1}{|\mathcal{I}_{\text{pitch}}|} \sum_{i \in \mathcal{I}_{\text{pitch}}} d_c(\hat{q}_i, q_i^{\text{ref}}). \quad (6)$$

Unresolved events are excluded from the denominator rather than counted as errors, and $|\mathcal{I}_{\text{pitch}}|$ is reported alongside every pitch number. Beyond pitch, we report string accuracy; left- and right-hand fingering precision, recall, and F1; ornament precision, recall, and F1; deterministic rule-violation rate; and technique-usage statistics to expose collapsed or overused techniques. Each violation is stored with an event ID, relevant state, and explanation. Rules that cannot yet be determined reliably are excluded from automatic scoring and listed as expert-review TODOs.

5 Results and Discussion

5.1 Comparisons and Ablations

We will compare direct generation, a single-stage interactive agent, and the two-stage GUQINAGENT. Ablations will remove pitch candidates, warning feedback, stateful audit, or compact context retrieval. We ask whether tools improve pitch correctness and rule compliance, whether warning feedback improves repair behavior without inducing unproductive loops, whether stage decomposition improves idiomatic notation, and whether inverse trajectories improve performance without simply increasing tool length.

6 Limitations, Ethics, and Resource Availability

6.1 Limitations and Ethics

The auditor is conservative and incomplete: no warning is not proof of musical correctness, and unresolved notation is not a success. Fine-grained left-hand reachability, timbre, and lineage-specific style still require expert review or additional physical modeling. The digitized score collection may not represent all historical sources or playing traditions. We position GUQINAGENT as an assistive research system, not a replacement for performers or tradition bearers. Before release, we will review source licenses and provenance, remove private teacher material, and publish only redacted trajectories intended for public supervision.

6.2 Open Science and Resource Availability

We will release the trained model, redacted training trajectories, the test benchmark, and the executable runtime used by teacher generation and inference. The release will include score-level split definitions, event-indexed schemas, evaluation scripts, filtering manifests, and instructions for reproducing the benchmark. Private teacher prompts, unredacted teacher reasoning, and any source material that cannot be redistributed will not be included. The archival repository URL, model license, dataset license, and versioned release identifier will be added before submission: [\[TODO: repository URL and licenses\]](#).

7 Conclusion

GUQINAGENT frames jianzipu generation as executable, stateful plan editing. Its benchmark makes score events, tuning, and notation state explicit; its two-stage architecture separates basic realization from idiomatic refinement; and its inverse trajectories create auditable public tool-use supervision from private annotations. Together, these components enable evaluation beyond text overlap and establish a practical basis for musically grounded notation agents.

References

- [1] Zehan Li, Fu Zhang, Zhijun Liu, and Jingwei Cheng. End-to-end structured information extraction from mixed-script documents in open compositional symbol systems. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2026.
- [2] Zehan Li, Fu Zhang, Zhijun Liu, and Jingwei Cheng. Zero-shot jianzi recognition as structured visual information extraction in open compositional symbolic systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29414–29429, 2026.
- [3] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

- [4] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.